

Refactoring CMS165

This topic discusses refactoring CMS165 - Controlling High Blood Pressure to use US Quality Core from the QI Core version.

The process follows the US Quality Core Update Process documented in the CMS Quality Measure Development guidance, but focuses on the final steps of refactoring to make use of common functions and elements available within the Using FHIR With CQL, CQL US Common, and US Quality Core implementation guides.

The discussion begins with a version of CMS165 that has already been refactored through steps 1 to 4 of the US Quality Core Update process.

NOTE: These are all proposed updates, identified to help improve the readability, maintainability, and implementability of the measure specification. Any accepted changes will be applied by the appropriate measure developer.

Consider .systolic() and .diastolic() functions

The expression for lowest readings use the following:

```
return singleton from(BPReading.component BPComponent
  where BPComponent.code ~ "Systolic blood pressure"
  return BPComponent.value as Quantity
```

This can be refactored to make use of the `.systolic()` function:

```
return BPReading.systolic().value
```

And the same for the `.diastolic()` function.

Consider Min/Max rather than First

The "Lowest Systolic/Diastolic Reading on Most Recent Blood Pressure Day" uses `First(... sort asc)` to mean "minimum." In CQL, nulls sort first ascending, so if any reading on that day is missing the component, the accessor yields null, and the result of the expression will be null. The result is that the patient will silently fail the numerator, even when valid readings exist:

```
define "Lowest Systolic Reading on Most Recent Blood Pressure Day":
  First("Qualifying Blood Pressure Reading" BPReading
    where BPReading.effective.latest() same day as "Most Recent Blood Pressure Day"
  return BPReading.systolic().value
```

```

    sort asc
  )

```

Consider using `Min`, which ignores nulls, but is also potentially more clearly communicates the semantic that the measure is looking for the minimum value.

```

define "Lowest Systolic Reading on Most Recent Blood Pressure Day":
  Min("Qualifying Blood Pressure Reading" BP
    where BP.effective.latest() same day as "Most Recent Blood Pressure Day"
    return BP.systolic().value
  )

```

The US Core BP profile does require both components, but the expressions are separate so the components can potentially be from different readings. With this approach `Min` degrades gracefully where `First(sort asc)` fails the numerator. The same consideration applies for "Most Recent Blood Pressure Day", where `Last(... sort asc)` is just `Max(...)`:

```

define "Most Recent Blood Pressure Day":
  Last("Blood Pressure Days" BPDays
    sort asc
  )

```

```

define "Most Recent Blood Pressure Day":
  Max("Blood Pressure Days")

```

Consider using an Encounter join

The following logic in the measure is doing two things:

```

define "Qualifying Blood Pressure Reading":
  ( ( ( [USCore.BloodPressureProfile] ).isObservationBP ( ) ) BloodPressure
    without ( ( [Encounter: "Encounter Inpatient"]
      union [Encounter: "Emergency Department Evaluation and Management Visit"]
    ).isEncounterPerformed ( ) ) DisqualifyingEncounter
    such that BloodPressure.effective.latest ( ) during day of
DisqualifyingEncounter.period
    where BloodPressure.effective.latest ( ) during day of "Measurement Period"
  )
  union ( ( ( [USCore.BloodPressureProfile] ).isObservationBP ( ) )
BloodPressure
    where ( not ( ( BloodPressure.encounter.getEncounter ( ) ).class.code in {
'EMER', 'IMP', 'ACUTE', 'NONAC', 'PRENC', 'SS' } ) )
    and BloodPressure.effective.latest ( ) during day of "Measurement

```

```
Period"
)
```

On the left side of the union, it's looking for blood pressure readings that did not take place on the *same day* as an emergency department or inpatient encounter (i.e. "acute" encounter).

On the right side of the union, it's bringing measurements back in that may have occurred on that same day, so long as they are not explicitly linked to an "acute" encounter, but using the `class` element of the encounter to determine that.

Focusing on the right side, first, the expression is using a fluent function to resolve the encounter:

```
define fluent function getEncounter(reference Reference):
  singleton from ( [Encounter] E where reference.references(E) )
```

But this is effectively running a full [Encounter] retrieve per blood pressure observation, and as noted before, that retrieve won't be cached by most engines (if any). Restructuring that as a join does it in one retrieve:

```
"Blood Pressure Readings" BloodPressure
with "Non Acute Encounters" E
such that BloodPressure.encounter.references(E)
```

Less impactful, but still an opportunity for improvement, `[USCore.BloodPressureProfile]` is retrieved twice (once per union branch); one named element definition fixes that.

This expression relies on the element refactorings for "Blood Pressure Readings", "Acute Care Encounters", and "Non Acute Encounters":

```
define "Blood Pressure Readings":
  ( [USCore.BloodPressureProfile] ).isObservationBP ( )

define "Acute Care Encounters":
  ( [Encounter: Concepts."Encounter Inpatient"]
    union [Encounter: Concepts."Emergency Department Evaluation and Management Visit"] ).isEncounterPerformed ( )

define "Non Acute Encounters":
  [Encounter] E
  where not ( exists ( "Acute Encounter Classes" AcuteClass
    where E.class ~ AcuteClass
  ) )
```

Looking specifically at "Non Acute Encounters", the original logic has:

```

        union ( ( ( [USCore.BloodPressureProfile] ).isObservationBP ( ) )
BloodPressure
        where ( not ( ( BloodPressure.encounter.getEncounter ( ) ).class.code in {
'EMER', 'IMP', 'ACUTE', 'NONAC', 'PRENC', 'SS' } ) )
        and BloodPressure.effective.latest ( ) during day of "Measurement
Period"
    )

```

In addition to the refactoring to use an element definition, consider defining a ValueSet for "Non Acute Care Setting Codes", rather than using a code-based comparison:

```

define "Non Acute Encounters":
    ( [Encounter: class in "Non Acute Care Setting Codes"] ).isEncounterPerformed (
    )

```

Note the flip in terminology usage here, we are proposing to characterize the non-acute care setting codes, rather than excluding encounters that are in an acute care setting. The binding is extensible so either approach may lead to missed data, but using the positive expression allows the terminology filter to be expressed inside the retrieve.

And, as illustrated above, consider using the `.isEncounterPerformed()` status function to ensure encounter state.

Consider renames to reflect measure intent

Some of the names of expressions in the logic could provide more communication about the intent. These are proposed based on inference from reading the logic, so they may not be quite right, but consider the following renames:

- "Qualifying Blood Pressure Reading" -> "Blood Pressure Reading Not From Acute Care"
- "Has Systolic Blood Pressure Less Than 140" -> "Systolic In Control"
- "Has Diastolic Blood Pressure Less Than 90" -> "Diastolic In Control"

Proposed Layering

Overall, these refactorings lead to the following proposed layering:

- CMS165Concepts: The value sets, and note the BP codes are no longer needed because we're using `.systolic()` and `.diastolic()`
- CMS165Elements: Which records count:
 - "Essential Hypertension Diagnoses"
 - "Pregnancy and Renal Diagnoses"
 - "End Stage Renal Disease Procedures"
 - "ERSD Monthly Outpatient Encounters"
 - "Acute Care Encounter"

- "Non Acute Encounters"
- "Blood Pressure Readings"
- CMS165Inferences: What they mean:
 - "Hypertension Diagnosed In First Six Months"
 - "Has PRegnancy Or Renal Diagnosis"
 - "Has End Stage Renal Disease Treatment"
 - "Blood Pressure Readings Not From Acute Care"
 - "Most Recent Blood Pressure Day"
 - "Lowest Systolic/Diastolic on Most Recent Day"
 - "Systolic/Diastolic In Control"
 - "Blood Pressure In Control"
- CMS165ControllingHighBPPProposed: Populations and SDEs only

The complete refactored logic is then:

CMS165ControllingHighBPPProposed

```

define "Initial Population":
  Patient.ageInYearsAt(date from end of "Measurement Period") in Interval[18, 85]
  and exists Inferences."Hypertension Diagnosed in First Six Months of
  Measurement Period"
  and exists AdultOutpatientEncounters."Qualifying Encounters"

define "Denominator":
  "Initial Population"

define "Denominator Exclusions":
  Hospice."Has Hospice Services"
  or Inferences."Has Pregnancy or Renal Diagnosis"
  or Inferences."Has End Stage Renal Disease Treatment"
  or AIFrallTCF."Is Age 66 to 80 with Advanced Illness and Frailty or Is Age 81
  or Older with Frailty"
  or AIFrallTCF."Is Age 66 or Older Living Long Term in a Nursing Home"
  or PalliativeCare."Has Palliative Care in the Measurement Period"

define "Numerator":
  Inferences."Blood Pressure In Control"

define "SDE Ethnicity":
  SDE."SDE Ethnicity"

define "SDE Payer":
  SDE."SDE Payer"

define "SDE Race":
  SDE."SDE Race"

```

```
define "SDE Sex":
  SDE."SDE Sex"
```

CMS165Inferences

Qualifying diagnosis

```
define "Hypertension Diagnosed in First Six Months of Measurement Period":
  Elements."Essential Hypertension Diagnoses" Hypertension
    where Hypertension.prevalenceInterval ( ) overlaps Interval[start of
"Measurement Period", start of "Measurement Period" + 6 months )
```

Exclusion inferences

```
define "Has Pregnancy or Renal Diagnosis":
  exists ( Elements."Pregnancy and Renal Diagnoses" Diagnosis
    where Diagnosis.prevalenceInterval ( ) overlaps "Measurement Period"
  )

define "Has End Stage Renal Disease Treatment":
  exists ( Elements."End Stage Renal Disease Procedures" ESRDProcedure
    where ESRDProcedure.performed.toInterval ( ) ends on or before end of
"Measurement Period"
  )
  or exists ( Elements."ESRD Monthly Outpatient Encounters" ESRDEncounter
    where ESRDEncounter.period starts on or before end of "Measurement Period"
  )
```

Eligible blood pressure readings

```
define "Blood Pressure Readings Not From Acute Care":
  "Blood Pressure Readings During Measurement Period Not On An Acute Care Day"
  union "Blood Pressure Readings During Measurement Period Linked To A Non Acute
Encounter"

define "Blood Pressure Readings During Measurement Period":
  Elements."Blood Pressure Readings" BloodPressure
    where BloodPressure.effective.latest ( ) during day of "Measurement Period"

define "Blood Pressure Readings During Measurement Period Not On An Acute Care
Day":
  "Blood Pressure Readings During Measurement Period" BloodPressure
    without Elements."Acute Care Encounters" AcuteEncounter
    such that BloodPressure.effective.latest ( ) during day of
AcuteEncounter.period
```

```

define "Blood Pressure Readings During Measurement Period Linked To A Non Acute Encounter":
  "Blood Pressure Readings During Measurement Period" BloodPressure
  with Elements."Non Acute Encounters" NonAcuteEncounter
  such that BloodPressure.encounter.references ( NonAcuteEncounter )

```

Blood pressure result

```

define "Blood Pressure Days":
  "Blood Pressure Readings Not From Acute Care" BloodPressure
  return date from BloodPressure.effective.latest ( )

define "Most Recent Blood Pressure Day":
  Max("Blood Pressure Days")

define "Lowest Systolic on Most Recent Blood Pressure Day":
  Min("Blood Pressure Readings Not From Acute Care" BloodPressure
    where BloodPressure.effective.latest ( ) same day as "Most Recent Blood Pressure Day"
  return FHIRHelpers.ToQuantity(BloodPressure.systolic ( ).value as Quantity)
  )

define "Lowest Diastolic on Most Recent Blood Pressure Day":
  Min("Blood Pressure Readings Not From Acute Care" BloodPressure
    where BloodPressure.effective.latest ( ) same day as "Most Recent Blood Pressure Day"
  return FHIRHelpers.ToQuantity(BloodPressure.diastolic ( ).value as Quantity)
  )

define "Systolic In Control":
  "Lowest Systolic on Most Recent Blood Pressure Day" < 140 'mm[Hg]'

define "Diastolic In Control":
  "Lowest Diastolic on Most Recent Blood Pressure Day" < 90 'mm[Hg]'

define "Blood Pressure In Control":
  "Systolic In Control"
  and "Diastolic In Control"

```

CMS165Elements

Conditions

```

define "Essential Hypertension Diagnoses":
  ( [Condition: Concepts."Essential Hypertension"] ).verified ( )

define "Pregnancy and Renal Diagnoses":
  ( [Condition: Concepts."Pregnancy"]

```

```
union [Condition: Concepts."End Stage Renal Disease"]
union [Condition: Concepts."Kidney Transplant Recipient"]
union [Condition: Concepts."Chronic Kidney Disease, Stage 5"] ).verified ( )
```

End stage renal disease treatment

```
define "End Stage Renal Disease Procedures":
  ( [Procedure: Concepts."Kidney Transplant"]
    union [Procedure: Concepts."Dialysis Services"] ).isProcedurePerformed ( )

define "ESRD Monthly Outpatient Encounters":
  ( [Encounter: Concepts."ESRD Monthly Outpatient Services"]
    ).isEncounterPerformed ( )
```

Encounters that disqualify a blood pressure reading

```
define "Acute Care Encounters":
  ( [Encounter: Concepts."Encounter Inpatient"]
    union [Encounter: Concepts."Emergency Department Evaluation and Management Visit"] ).isEncounterPerformed ( )

define "Non Acute Encounters":
  ( [Encounter: class in "Non Acute Care Setting Codes"] ).isEncounterPerformed ( )
```

Blood pressure

```
define "Blood Pressure Readings":
  ( [USCore.BloodPressureProfile] ).isObservationBP ( )
```